

Employee Attrition Prediction & Workplace Analytics

— Problem Statement, Data Dictionary, and Analytical Questions

1. Problem Statement

1.1 Business Context

Employee attrition (voluntary or involuntary turnover) is a critical business problem across industries. Replacing an employee typically costs 50% to 200% of their annual salary in recruitment, onboarding, lost productivity, and knowledge loss. Organizations that can predict and understand why employees leave are better positioned to design retention programs, adjust compensation, improve work-life balance, and reduce unwanted turnover.

This capstone project uses a large, realistic human resources dataset to build end-to-end machine learning solutions that address:

- Predicting which employees are at risk of leaving (classification).
- Identifying key drivers of attrition (feature importance, EDA).
- Segmenting employees into meaningful groups for targeted interventions (clustering).
- Understanding relationships between satisfaction, tenure, income, and attrition (regression/interpretation).

1.2 Real-World Significance

The problem has direct real-world significance: HR and people analytics teams use similar datasets and ML models to prioritize retention efforts, allocate training budgets, and design diversity and inclusion initiatives. Outcomes of this project mirror industry practice: binary attrition prediction, segment-based action plans, and interpretable drivers.

1.3 ML Problem Types Addressed

Supervised Learning — Classification: Predict attrition (Yes/No) using demographic, job, and satisfaction features. Metrics: accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix.

Supervised Learning — Regression (optional): Predict monthly income or job satisfaction score from other features to understand compensation and satisfaction drivers.

Unsupervised Learning — Clustering: Segment employees (e.g., K-Means, hierarchical) to discover at-risk or high-value groups without using the attrition label.

1.4 Project Objectives

Primary: Build a classification model that predicts employee attrition (Yes/No) with measurable accuracy and interpretability. Secondary: Perform exploratory data analysis to identify drivers of attrition; apply clustering to segment employees for targeted retention; optionally build a regression model to predict income or satisfaction. Learning: Demonstrate the full ML pipeline (data prep, feature engineering, model training, evaluation, deployment-ready documentation).

1.5 Success Criteria

Model performance: Classification model achieves F1-score and AUC-ROC above a reasonable baseline (e.g., $F1 > 0.55$, $AUC > 0.65$). Interpretability: Top 5–10 features driving attrition are identified and explained in business terms. Clustering: At least one clustering solution (e.g., K-Means) with k chosen by elbow/silhouette; segments described and linked to attrition where applicable. Documentation: Jupyter notebook and report include problem statement, EDA, preprocessing steps, model comparison, and actionable recommendations.

1.6 Assumptions and Constraints

Assumptions: Data is representative of the organization; attrition label is accurate; no major data leakage (e.g., `join_date` used only for tenure, not future info). Constraints: Use only provided features; no external data unless justified; models must be trainable on standard hardware (e.g., 20k rows, <30 features). Ethical: Results are for learning and internal HR analytics; no discriminatory use of protected attributes (gender, age) in final decision logic without fairness analysis.

1.7 Stakeholders and Use Cases

Stakeholders: HR leadership (retention strategy), people analytics team (model deployment), line managers (targeted retention actions). Use cases: (1) Score current employees for attrition risk and prioritize retention outreach; (2) Identify which levers (satisfaction, pay, workload) to improve; (3) Segment workforce for differentiated policies (e.g., high-risk segment gets extra training or compensation review).

1.8 Scope and Deliverables

Scope: Data loading and exploration; missing value and encoding handling; train-test split; at least one classification model (e.g., logistic regression, decision tree, or random forest); evaluation with appropriate metrics; optional regression and clustering; feature importance and interpretation; final report with problem statement, EDA summary, model results, and business recommendations.

Deliverables (code): Solution must be implemented in Python (.py files or Jupyter notebooks). Recommended structure: `solution_01_ed.py` (load, EDA, plots), `solution_02_preprocessing.py` (ColumnTransformer, train_test_split),

solution_03_classification.py (LogisticRegression, DecisionTree, RandomForest, metrics, confusion matrix, ROC), solution_04_clustering.py (KMeans, elbow, silhouette, attrition by cluster), solution_05_regression.py (income prediction, R^2 , RMSE). A run_full_solution.py may orchestrate the full pipeline. All plots saved to an output/ directory.

2. Data Dictionary

The dataset contains 20,000 employee records and 29 variables. Below is the data dictionary.

Variable Name	Data Type	Description	
employee_id	Integer	Unique identifier for each employee.	
age	Integer	Age of the employee in years (18–60).	
gender	Categorical	Male / Female.	
marital_status	Categorical	Single / Married / Divorced.	
education	Ordinal (1–5)	1=Below College, 2=College, 3=Bachelor, 4=Master, 5=Doctorate.	
education_field	Categorical	Life Sciences, Medical, Marketing, Technical Degree, HR, Other.	
department	Categorical	Sales, R&D, HR, Finance, Marketing, Operations.	
job_role	Categorical	Specific role title (e.g., Sales Executive, Research Scientist).	
job_level	Ordinal (1–5)	Job level within the organization.	
business_travel	Categorical	Travel_Rarely / Travel_Frequently / Non-Travel.	
distance_from_home_km	Integer	Distance from home to office in km (1–29).	
num_companies_worked	Integer	Number of companies the employee has worked for (0–9).	

years_at_company	Integer	Total years with the current company (0-40).	
years_in_current_role	Integer	Years in the current job role.	
years_since_last_promotion	Integer	Years since last promotion (0-15).	
years_with_current_manager	Integer	Years with the current manager (0-17).	
monthly_income_usd	Integer	Monthly income in USD (1500-20000).	
stock_option_level	Ordinal (0-3)	Stock option level (0=none to 3=highest).	
num_projects_last_year	Integer	Number of projects worked on in the last year (2-7).	
training_times_last_year	Integer	Number of training sessions attended (0-6).	
environment_satisfaction	Ordinal (1-4)	1=Low to 4=High.	
job_satisfaction	Ordinal (1-4)	1=Low to 4=High.	
work_life_balance	Ordinal (1-4)	1=Low to 4=High.	
job_involvement	Ordinal (1-4)	1=Low to 4=High.	
relationship_satisfaction	Ordinal (1-4)	1=Low to 4=High.	
performance_rating	Ordinal (3-4)	3=Good, 4=Excellent (company scale).	
overtime	Binary	Yes / No.	
attrition	Binary (Target)	Yes = left company, No = stayed.	
join_date	Date	Date of joining the company (YYYY-MM-DD).	

3. Technical Stack & Implementation Requirements

Libraries: pandas (data load, EDA, manipulation), numpy (arrays, numeric ops), scikit-learn (preprocessing, models, metrics), matplotlib and seaborn (visualization). Python 3.8+.

Dataset: capstone_employee_attrition_dataset.csv (20,000 rows, 29 columns).

3.1 Code-Oriented Requirements

Data loading: Use `pd.read_csv()` to load the dataset; implement a function that returns `(X, y)` or `(df, target_col)` and excludes `employee_id`, `join_date` from features.

Preprocessing: Use sklearn ColumnTransformer with StandardScaler for numeric columns and OneHotEncoder(drop="first") for categorical columns. Use train_test_split(X, y, test_size=0.2, stratify=y, random_state=42) so the target class distribution is preserved in train and test.

Classification: Instantiate and fit at least LogisticRegression, DecisionTreeClassifier, and RandomForestClassifier from sklearn. Use classification_report(), confusion_matrix(), roc_auc_score(), and plot ROC curves with sklearn.metrics.roc_curve(). Handle class imbalance via class_weight="balanced" or SMOTE from imblearn.

Clustering: Use sklearn.cluster.KMeans; compute inertia for k in range(2, 11) and plot elbow curve; compute silhouette_score for each k. Fit final KMeans with chosen k and assign cluster labels; compute attrition rate per cluster using groupby().

Regression: Use LinearRegression and RandomForestRegressor to predict monthly_income_usd; report r2_score and mean_squared_error (or RMSE).

4. Technical & Code-Oriented Questions

Answer these using code (e.g., in Jupyter or .py files). Where applicable, write the actual Python/sklearn calls and interpret the output.

4.1 Data Loading and EDA (Code)

1. Write code to load the CSV with pandas and print df.shape, df.dtypes, and df.isnull().sum(). What is the shape and are there any missing values?
2. Using df["attrition"].value_counts(normalize=True) and groupby("department")["attrition"].apply(lambda x: (x=="Yes").mean()), compute overall attrition rate and attrition rate by department. Plot both with matplotlib or seaborn.
3. Create a correlation matrix of all numeric columns plus a binary attrition column (Yes=1, No=0). Which three numeric features have the highest absolute correlation with attrition? Use df.corr() and sort by correlation with target.
4. Use df.boxplot(column="monthly_income_usd", by="attrition") and similarly for overtime. Write one line to compute attrition rate by overtime (Yes/No).
5. List the column names that are of type object (categorical). For OneHotEncoder, how many feature columns will you get after encoding if you use drop="first"? (Count manually or use OneHotEncoder().fit(df[cat_cols]).get_feature_names_out().size)

4.2 Preprocessing and Train-Test Split (Code)

6. Implement a ColumnTransformer with ("num", StandardScaler(), num_cols) and ("cat", OneHotEncoder(drop="first"), cat_cols). Fit it on X_train and transform both X_train and X_test (never fit on test). Write the code.
7. Why must you use stratify=y in train_test_split for this dataset? What happens to the proportion of attrition=Yes in train and test if you omit stratify?

8. After preprocessing, how many columns does `X_train` have? (Hint: `num_cols` count + number of `OneHotEncoder` output columns.) Write code to print `X_train.shape`.

4.3 Classification (Code)

9. Fit `LogisticRegression(max_iter=1000, class_weight="balanced")` on `(X_train, y_train)`. Call `precision_score`, `recall_score`, `f1_score`, `roc_auc_score` on `(y_test, y_pred)` and `(y_test, y_pred_proba)`. Write the code and report the four metrics.
10. Using `sklearn.metrics.confusion_matrix(y_test, y_pred)`, plot a heatmap with `seaborn`. In business terms, what do false positives and false negatives represent for the attrition problem?
11. Fit a `RandomForestClassifier` and extract `model.feature_importances_`. Pair with feature names (from `ColumnTransformer.get_feature_names_out` or manual list) and sort descending. List the top 5 features. Write the code.
12. Implement `GridSearchCV(LogisticRegression(), param_grid={"C": [0.01, 0.1, 1, 10]}, cv=5, scoring="f1")` and report the best C and best F1. Write the code.

4.4 Clustering (Code)

13. For `k` in `range(2, 11)`, fit `KMeans(n_clusters=k, random_state=42)` on scaled numeric features and append inertia to a list. Plot `k` vs inertia (elbow curve). Which `k` would you pick from the plot? Write the code.
14. Compute `silhouette_score(X_scaled, labels)` for each `k`. Plot `k` vs silhouette score. Does the best `k` from silhouette match the elbow? Write the code.
15. After fitting `KMeans` with your chosen `k`, add cluster labels to the `DataFrame`. Compute `df.groupby("cluster")["attrition"].apply(lambda x: (x=="Yes").mean())`. Which cluster has the highest attrition rate? Write the code.

4.5 Regression (Code)

16. Build `X` (all features except `employee_id`, `join_date`, `attrition`, `monthly_income_usd`) and `y = df["monthly_income_usd"]`. Preprocess with the same `ColumnTransformer` approach. Fit `LinearRegression` and `RandomForestRegressor`. Report `r2_score` and `np.sqrt(mean_squared_error(y_test, y_pred))` for both. Write the code.
17. From the regression model, which feature (if using linear model, use `model.coef_`) is most associated with higher `monthly_income_usd`? How do you interpret the coefficient in dollars?

4.6 Pipeline and Reproducibility

18. Write a single script or notebook that runs in order: `load_data()` -> `eda_plots()` -> `preprocess_split()` -> `fit_classification_models()` -> `fit_kmeans()` -> `fit_regression()`. Save all plots to an output/ folder. Provide the high-level code structure.
19. How would you use `sklearn.pipeline.Pipeline` to chain `StandardScaler` and `LogisticRegression` so that you can fit once and reuse on new data without refitting the scaler? Write a 3-line example.

— End of Problem Statement & Data Dictionary —